

We aim to solve the linear chance constraint program

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & P_F(\xi^T x \leq b) \geq 1 - \alpha \end{aligned} \tag{1}$$

where $c \in \mathbb{R}^d, b \in \mathbb{R}$ are deterministic and known, $\xi \in \mathbb{R}^d$ is a random vector whose true distribution F is unknown, and $1 - \alpha$ is a given safety level (e.g., 95%). P_F denotes the probability under the true distribution F . The program seeks to minimize the cost $c^T x$ subject to the constraint that some safety condition $\xi^T x \leq b$ is satisfied with a probability $\geq 1 - \alpha$.

We consider the context where i.i.d. data $\xi_1, \dots, \xi_n$ are available from F , and would like to use the data to find a solution $\hat{x}$ of (1) that, firstly, satisfies the chance constraint with a probability at least $1 - \beta$ (say 95%), or formally, $P_{\text{data}}(P_F(\xi^T \hat{x} \leq b) \geq 1 - \alpha) \geq 1 - \beta$; and secondly, attains an as low cost $c^T \hat{x}$ as possible.

Code “True_sol.m”. This code is to compute the true optimal solution of (1). This only works for multivariate Gaussian ξ .

1 Existing Methods

Existing methods for solving (1) can be grouped into two categories. The first category consists of the methods that do not use data, but build on some population-level information. The second category consists of data-driven methods that use data to find a solution.

1.1 Non-Data-Driven Methods

1. Safe convex approximation (SCA). Code “SCA.m” Since (1) can be non-convex, this method approximate (1) by a convex program whose feasible set is a subset of the feasible set of (1), and then solve the convex approximation. The resulting solution is guaranteed feasible (i.e., $1 - \beta = 1$). See Section 2.4 of Ben-Tal et al. (2009) for more details. To run SCA, execute “SCA.m” by supplying c, b, α , and the **true** mean and covariance matrix of ξ . This code “SCA.m” only works for ξ that is multivariate Gaussian.
2. Distributionally robust optimization (DRO) with moment information. Code “DRO1.m”. Given the true mean μ and true covariance matrix Σ of ξ , this method replaces the chance constraint in (1) by

$$\inf_{Q \text{ s.t. } E_Q[\xi] = \mu, \text{Cov}_Q(\xi) = \Sigma} P_Q(\xi^T x \leq b) \geq 1 - \alpha.$$

Like in SCA, the resulting solution is guaranteed feasible. See Theorem 1 in Ghaoui et al. (2003) for more details, and you need to supply the true mean and covariance matrix to “DRO1.m”.

1.2 Data-Driven Methods

1. Distributionally robust optimization (DRO) with uncertain moment information. Code “DRO2.m”. The difference from “DRO1.m” is that now the mean and covariance matrix are estimated

from data. Let $\mathcal{U}_\rho$ be a $(1 - \beta)$ -level confidence region for (μ, Σ) that is computed from the data. This method replaces the chance constraint in (1) by

$$\inf_{Q \text{ s.t. } (E_Q[\xi], \text{Cov}_Q(\xi)) \in \mathcal{U}_\rho} P_Q(\xi^T x \leq b) \geq 1 - \alpha.$$

See Section 5 in Marandi et al. (2017) for more details on $\mathcal{U}_\rho$. “DRO2.m” uses a ρ value from equation (19) in Marandi et al. (2017). In this method, ρ is the parameter.

2. Robust optimization (RO) based on quantile estimation (Hong et al. (2017)). Code “RO.m”. This method splits the data into two portions. The first portion is used to learn the shape of the uncertainty set $\mathcal{U}_t$ for ξ , and then the second portion is used to calibrate the size t of the set such that $P_{\text{data}}(P_F(\xi \in \mathcal{U}_t) \geq 1 - \alpha) \geq 1 - \beta$. Then the chance constraint in (1) is replaced by

$$\sup_{\xi \in \mathcal{U}_t} \xi^T x \leq b.$$

“RO.m” implements the method with $\mathcal{U}_t$ being an ellipsoid and $\sqrt{t}$ being its radius. In this method, t is the parameter.

3. Robust optimization based on reconstruction (An improvement of “RO.m” in Section 5 of Hong et al. (2017)). Code “RO_reconstruct.m”. In this method, also t is the parameter.
4. Scenario generation (SG). Code “SG.m”. Given the data set $\xi_1, \dots, \xi_n$, this method simply solves

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & \xi_i^T x \leq b \text{ for } i = 1, \dots, n \end{aligned}$$

Equation (5) from Carè et al. (2014) tells how large n needs to be in order to produce a feasible solution with $1 - \beta$ confidence. In this method, the number of constraints to be satisfied can be a parameter. The optimization with the first s constraints satisfied is

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & \xi_i^T x \leq b \text{ for } i = 1, \dots, s \end{aligned}$$

5. Fast Scenario generation (Carè et al. (2014)). Code “SG_fast.m”. This method divides the data into two portions. Use the first portion to run an SG to obtain a solution x_1 , and use the second portion to run another SG on the line segment $(1 - \lambda)x_1$ for $\lambda \in [0, 1]$. In this method, $\lambda \in [0, 1]$ is the parameter.

2 Our Method

Our method, Algorithm 1, divides data into two portions. The reformulation in phase one can be any data-driven method discussed in Section 1.2. Some implementation details:

- When none of $s_j, j = 1, \dots, m$ satisfies the criterion in (2), still output some reasonable solution, e.g., a solution $x^*(s_j)$ with the largest $\hat{P}(x^*(s_j)) - z_{1-\beta} \frac{\hat{\sigma}(x^*(s_j))}{\sqrt{n_2}}$. Similar modifications apply to Method 2, 3 and 4.
- In Method 2 and Method 3, when all $\hat{\sigma}^2(x^*(s_j))$ are zero, just let $\hat{q}_{1-\beta}$ or $\hat{q}_{1-\beta}^u$ be 0.

Algorithm 1 Parameter Calibration via Marginal Gaussian

Input: n observations $\xi_{1:n} = \{\xi_1, \dots, \xi_n\}$; numbers of observations n_1, n_2 allocated to each phase such that $n_1 + n_2 = n$; a confidence level $1 - \beta$; a data-driven reformulation with a (possibly multi-dimensional) parameter s that lies in some space $\mathcal{S}$.

Procedure:

Phase one:

Use the first n_1 observations, $\{\xi_1, \dots, \xi_{n_1}\}$, to construct the data-driven reformulation of (1) parametrized by $s \in \mathcal{S}$. Discretize S as $\{s_1, \dots, s_m\}$, and solve the reformulated problem to obtain the corresponding solutions $\{x^*(s_j) | j = 1, \dots, m\}$.

Phase two:

1. For each solution $x^*(s_j)$, use the remaining n_2 observations, $\{\xi_{n_1+1}, \dots, \xi_n\}$, to compute the sample mean $\hat{P}(x^*(s_j)) = \frac{1}{n_2} \sum_{i=1}^{n_2} \mathbf{1}(\xi_{n_1+i}^T x^*(s_j) \leq b)$ and the sample variance $\hat{\sigma}^2(x^*(s_j)) = \hat{P}(x^*(s_j))(1 - \hat{P}(x^*(s_j)))$.

Method 1:

2. Compute an

$$s^* \in \operatorname{argmin} \left\{ f(x^*(s_j)) \mid \hat{P}(x^*(s_j)) \geq 1 - \alpha + z_{1-\beta} \frac{\hat{\sigma}(x^*(s_j))}{\sqrt{n_2}}, j = 1, \dots, m \right\} \quad (2)$$

where $z_{1-\beta}$ is the $1 - \beta$ quantile of the standard normal.

Method 2:

2. Compute the sample covariance matrix $\hat{\Sigma}$ with $\hat{\Sigma}(j_1, j_2) = \frac{1}{n_2} \sum_{i=1}^{n_2} [\mathbf{1}(\xi_{n_1+i}^T x^*(s_{j_1}) \leq b) - \hat{P}(x^*(s_{j_1}))][\mathbf{1}(\xi_{n_1+i}^T x^*(s_{j_2}) \leq b) - \hat{P}(x^*(s_{j_2}))]$.

3. Compute the $(1 - \beta)$ -quantile, $\hat{q}_{1-\beta}$, of

$$\max \left\{ Z_j / \hat{\sigma}(x^*(s_j)) \mid j = 1, \dots, m, \hat{\sigma}^2(x^*(s_j)) > 0 \right\}$$

where $(Z_1, \dots, Z_m)$ follows multivariate normal with mean zero and covariance $\hat{\Sigma}$, and let

$$s^* \in \operatorname{argmin} \left\{ f(x^*(s_j)) \mid \hat{P}(x^*(s_j)) \geq 1 - \alpha + \hat{q}_{1-\beta} \frac{\hat{\sigma}(x^*(s_j))}{\sqrt{n_2}}, j = 1, \dots, m \right\}.$$

Method 3:

2. Do the same as Method 2 Step 2.

3. Compute the $(1 - \beta)$ -quantile, $\hat{q}_{1-\beta}^u$, of

$$\max \left\{ Z_j \mid j = 1, \dots, m, \hat{\sigma}^2(x^*(s_j)) > 0 \right\}$$

where $(Z_1, \dots, Z_m)$ follows multivariate normal with mean zero and covariance $\hat{\Sigma}$, and let

$$s^* \in \operatorname{argmin} \left\{ f(x^*(s_j)) \mid \hat{P}(x^*(s_j)) \geq 1 - \alpha + \frac{\hat{q}_{1-\beta}^u}{\sqrt{n_2}}, j = 1, \dots, m \right\}.$$

Method 4:

2. Compute an

$$s^* \in \operatorname{argmin} \left\{ f(x^*(s_j)) \mid \hat{P}(x^*(s_j)) \geq 1 - \alpha, j = 1, \dots, m \right\}.$$

Output: the solution $x^*(s^*)$.

3 What to Implement

1. Choose some multivariate Gaussian distribution for ξ , and choose appropriate values of c, b for the problem. Use “True_sol.m” to obtain the true solution.
2. Without data, implement the non-data-driven methods in Section 1.1.
3. With data $\xi_1, \dots, \xi_n$, compare the performance of all data driven methods in Section 1.2 with our method. Report the average objective value, average safety level and confidence level. When comparing to method X from Section 1.2, in our algorithm the reformulation in Phase one should be the method X .
4. Repeat 3 under different data size n and different dimension d of x (which is equal to the dimension of ξ in this case).

Some notes/suggestions:

1. In case cvx does not successfully solve an optimization problem, the current code would return *NaN* as a solution.
2. Look at the parameter values chosen by each existing method, in order to get a sense of what discrete parameter values $s_1, \dots, s_m$ to use in our algorithm. In the SG case, this is clear, just use $s_i = i$ for $i = 1, \dots, n_1$.

References

- Ben-Tal A, El Ghaoui L, Nemirovski A (2009) *Robust optimization*, volume 28 (Princeton University Press).
- Carè A, Garatti S, Campi MC (2014) Fast—fast algorithm for the scenario technique. *Operations Research* 62(3):662–671.
- Ghaoui LE, Oks M, Oustry F (2003) Worst-case value-at-risk and robust portfolio optimization: A conic programming approach. *Operations research* 51(4):543–556.
- Hong LJ, Huang Z, Lam H (2017) Learning-based robust optimization: Procedures and statistical guarantees. *arXiv preprint arXiv:1704.04342* .
- Marandi A, Ben-Tal A, den Hertog D, Melenberg B (2017) Extending the scope of robust quadratic optimization. *Available on Optimization Online* .